Assignment 2 - Interplanetary Transfer

Last update: 3 March 2026

For this assignment, you will use a Lambert targeter to generate a first guess for an unperturbed interplanetary transfer. Subsequently, you will use various numerical propagation models that include perturbations to analyze the trajectory in higher fidelity, and compute trajectory corrections such that your trajectory meets its boundary conditions (at departure and arrival) in the perturbed environment.

In this assignment, you are required to modify five Python files. Unlike assignment 1, this assignment provides you with an uncompleted .py file for each question, as well as a file containing a number of helper functions, some of which you are to complete. To retrieve the code from Github, navigate to your local git repository using your terminal (*e.g* to the directory containing your Assignment1 directory), and use the following commands (if the first line yields an error, it likely means that the central repository for the assignments is already set correctly, and you can continue with the other command lines):

```
git remote add upstream https://github.com/tudat-team/numerical-astrodynamics-2026
git fetch upstream
git restore --source upstream/main Assignment2/
git add Assignment2
git commit -m "Updated code for assignment 2"
```

Doing so should add the Assignment2 directory, with five .py files, and an empty `SimulationOutput` directory.

General Instructions

In this assignment, you will use a so-called Lambert targeter (a tool that solves Lambert's problem) to generate an initial guess for an interplanetary direct high-thrust transfer. The Lambert targeter takes as input:

- A departure position $\mathbf{r}_0$
- An arrival position $\mathbf{r}_E$
- A time of flight T
- A central body gravitational parameter μ

The Lambert targeter then generates the Keplerian trajectory between $\mathbf{r}_0$ and $\mathbf{r}_E$, with the given time of flight T. The initial and final positions uniquely define the trajectory (for a given T; assuming a prograde single-revolution trajectory; ignoring rare singular cases), so the full state along this Keplerian trajectory (ie. Lambert arc) are *outputs* of the Lambert targeter. We denote the Cartesian state function of this Lambert arc as $\bar{\mathbf{x}}(t)$. For your situation, the initial and final position $\mathbf{r}_0$ and $\mathbf{r}_E$ of the full trajectory are the positions of the center of mass of Earth and Mars or Venus (w.r.t. the Sun), at times t_0 and t_E, respectively (NB: which you should use depends on your student number; find this in the `assignment2Input-2026.txt` file on Brightspace under Assignment

2, with t_0 and t_E in Julian days).

All analysis on the output data can be done in the Python code. However, if you would like to use a different piece of software (*e.g.* Matlab) for your analyses, the relevant data is provided as output to data files:

- By calling the `write_propagation_results_to_file` function, the propagated state of the spacecraft and the associated dependent variables will be saved to a file, as well as the state of the Lambert arc $\bar{\mathbf{x}}$ at each epoch of the numerical integration. See the in-code comments of this function for more details.

Before starting the assignment, read the submission instructions given at the end of this document.

1. **(10 points; Maximum text length: 5 lines)**
Use the Lambert arc's initial state $\bar{\mathbf{x}}(t_0)$ as initial state for the numerical propagation (spacecraft w.r.t. Sun), with the given initial time t_0. Run the code to propagate the state of the spacecraft using only the Sun's point-mass attraction, with the Sun as propagation origin.

Answer the following question:

Create the following plots:

- The total trajectory in three dimensions.
- The difference between the Lambert targeter result $\bar{\mathbf{x}}(t)$ and the numerical propagation $\mathbf{x}(t)$. Specifically, plot the difference in each of the three Cartesian position components of the spacecraft w.r.t. the Sun.

And answer the following question:

- Provide a comprehensive list of the assumptions of the Lambert arc model. State and briefly explain for each one whether it is true in your numerical propagation. If needed, back up your argumentation by adding, and plotting, any additional dependent variables to the simulation output you see fit. For each assumption that you state is not true in the numerical propagation, explain whether the difference is relevant for your numerical propagation.

Add to save file 1: Row 1: initial propagation time and Cartesian state.
Add to save file 1: Row 2: final propagation time and Cartesian state.

Coding instructions and hints:

The code provided for question 1 should propagate the dynamics for *unperturbed* dynamics, with the initial state extracted directly from the Lambert arc. The resulting numerical state history ($\mathbf{x}(t)$) is stored in the `state_history` variable. The state history, as computed directly from the Lambert arc ($\bar{\mathbf{x}}(t)$) is stored directly in the `lambert_history` variable.

To make this function generate results, you have a **Code task** for the following helper functions:

- `create_simulation_bodies`: this function defines the body settings and creates the body objects
- `get_unperturbed_propagator_settings`: this function defines the propagator settings for the unperturbed case

In order to ensure compatibility with the rest of the code, name your vehicle: `"Spacecraft"`.

2. **(20 points; Maximum text length: 8 lines)**
Now add the following perturbations to the numerical propagation (see coding instructions and hints at end of this question):

- Point-mass gravity by Venus, Earth, Moon, Mars, Jupiter, Saturn, and the Sun
- Cannonball radiation pressure on the spacecraft. Use a reference area of 20 m^2, a radiation pressure coefficient of 1.2, and a vehicle mass of 1000 kg.

To do so, also update the definition of the environment so that these accelerations can be evaluated.

Run the propagation for four cases:

i The initial and final propagation time equal to the initial and final times of the Lambert arc.

ii The initial and final propagation time shifted forward and backward in time, respectively, by $\Delta t = 1$ hour.

iii The initial and final propagation time shifted forward and backward in time, respectively, such that the propagation starts/ends *on* the sphere of influence of the Earth and Mars/Venus, respectively, w.r.t. the Sun. Retrieve the initial propagation time that meets this condition from the Lambert arc. For the propagation termination, in the case where the perturbations are so strong that the spacecraft never enters the sphere of influence of the target body, terminate the propagation on the final time of the Lambert arc. [1]

iv Same as (ii), but now propagating forward and backward from the middle point (in time) of the propagation.[2]

Note that if you shift the initial time from t_0 to $t_0+\Delta t$ (by modifying these inputs to the `propagate_trajectory` function), the initial state used for the propagation will be adjusted (automatically) accordingly to $\bar{\mathbf{x}}(t_0+\Delta t)$.

Answer the following questions:

(a) For each propagation, plot the quantities [3]
$\Delta r = ||\mathbf{r}(t) - \bar{\mathbf{r}}(t)||$, $\Delta v = ||\mathbf{v}(t) - \bar{\mathbf{v}}(t)||$ and $\Delta a = ||\mathbf{a}(t) - \bar{\mathbf{a}}(t)||$ as a function of time.

(b) Explain why Δr behaves very differently in the cases that are analyzed in (a). Specifically, explain the following points, using the plots from (a) to support your answers:

- Why increasing the buffer time Δt seems to generally reduce the magnitude of Δr
- Why starting the propagation at middle of the arc, and prop-

[1] Use the `dynamics.propagation_setup.propagator.hybrid_termination` function to define the termination conditions

[2] Use the `dynamics.propagation_setup.propagator.non_sequential_termination` function to define the termination conditions

[3] The quantity $\bar{\mathbf{a}}(t)$ is not provided automatically, compute its value at each step yourself.

agating forwards and backwards, results in a smaller maximum value Δr than only propagating forward, even for equal Δt (case iv vs. case ii)

(c) Repropagate case (iv) two more times, once with a vehicle mass of 500 kg and once with a vehicle mass of 250 kg. Indicate how this will influence your equation of motion. For both propagations, plot the difference in position w.r.t. the results for case (iv) you obtained in (2b). Using the three variants of case (iv) (with the three different masses), define and compute a figure of merit from which you can assess the linearity of the effect of the mass. Using this figure of merit, discuss whether you consider the impact of mass on position to be linear. If you do, discuss a condition under which this linearity would break. If not, discuss a reason for the non-linearity.

Add to save file 1: row 3: initial propagation time and Cartesian state (case i).
Add to save file 1: row 4: final propagation time and Cartesian state (case i).
Add to save file 1: row 5: initial propagation time and Cartesian state (case ii, question a).
Add to save file 1: row 6: final propagation time and Cartesian state (case ii, question a).
Add to save file 1: row 7: initial propagation time and Cartesian state (case iii).
Add to save file 1: row 8: final propagation time and Cartesian state (case iii).

Coding instructions and hints

The code for question 2 should propagate the orbit for *perturbed* dynamics, with the initial state extracted directly from the Lambert arc. The resulting numerical state history ($\mathbf{x}(t)$) is stored in the `state_history` variable. On the other hand, the state history as computed directly from the Lambert arc ($\bar{\mathbf{x}}(t)$) is stored directly in the `lambert_history` variable.

To make the code generate the required results, you have a **Code task** for the following helper functions:

- `create_simulation_bodies`: this function defines the body settings and creates the body objects
- `get_perturbed_propagator_settings`: this function defines the propagator settings for the perturbed case

To propagate the dynamics, do not create a `SingleArcSimulator` manually, but make use of the `propagate_trajectory` function (which creates a `SingleArcSimulator` for you).

3. **(35 points; Maximum text length: 12 lines)**
We will now turn our attention to modifying the trajectory so that we arrive at the desired target also when *including* perturbations.

When we modify an initial state $\mathbf{x}_0$ or some parameter vector $\mathbf{p}$ (which contains properties of the dynamical model), the propagated state $\mathbf{x}(t)$ will change. When changing the initial state by $\Delta\mathbf{x}_0$ and/or a parameter vector by $\Delta\mathbf{p}$, we will denote the resulting change in numerically propagated state, as a function of time, as $\Delta\mathbf{x}(t)$.

For given modifications $\Delta\mathbf{x}_0$ and $\Delta\mathbf{p}$, the behaviour of $\Delta\mathbf{x}(t)$ can be obtained by directly numerically repropagating twice, once by propagating $\mathbf{x}(t)$ with the modified initial state and parameter vector, and once with the unmodified values, and then subtracting the two sets of results. However, this can be a tedious and computationally expensive process. The matrices $\mathbf{\Phi}(t, t_0)$ and $\mathbf{S}(t)$ provide a way to approximate $\Delta\mathbf{x}(t)$, by means of a *linearization*:

$$\mathbf{\Phi}(t, t_0) = \frac{\partial \mathbf{x}(t)}{\partial \mathbf{x}_0} \tag{1}$$

$$\mathbf{S}(t) = \frac{\partial \mathbf{x}(t)}{\partial \mathbf{p}} \tag{2}$$

Now we denote the linearized change in state as $\Delta\tilde{\mathbf{x}}(t)$, which we compute from:

$$\Delta\tilde{\mathbf{x}}(t) = \mathbf{\Phi}(t, t_0)\Delta\mathbf{x}_0 + \mathbf{S}(t)\Delta\mathbf{p} \tag{3}$$

so that $\Delta\tilde{\mathbf{x}}(t)$ is a *linear approximation* of $\Delta\mathbf{x}(t)$.

For this question, we split the total propagation time into 10 equisized (in time) arcs. Using the propagation settings of the previous question (case iii), the state and variational equations for the dynamics are propagated for each arc. For each arc i (with i starting at 0), the initial state $\mathbf{x}_i(t_{0,i})$ is set so that it corresponds exactly with the state proscribed by the Lambert targeter at the corresponding time $\bar{\mathbf{x}}(t_{0,i})$. This will result in a discontinuous state history over the full transfer (since the propagation is performed in an arc-wise manner). For this question, you will compute the correction maneuvers necessary to make the trajectory continuous in position; you will do this by applying an impulsive correction maneuver at the start of each arc, such that $\mathbf{r}_i = \bar{\mathbf{r}}_i$ at the end of each arc.

Notes on figures: make sure that the figures you make are clear, and allow the information that you want to convey to be observed. Specifically consider:

- Can the behaviour at small values of the quantity on the vertical axis be seen in your figure? If not, and if this is relevant to the question, consider using a logarithmic scale.
- Can all curves be distinguished (e.g. are curves overlapping)? If so, modify the line styles (for instance) so that a dashed line falls on top

of a solid line (with different colors) rather than two superimposed solid lines.

- Make sure all text (legend, axes, etc.) are clearly legible without zooming.

Answer the following questions:

(a) Perform the arcwise propagation. Plot the deviation $\Delta r(t)$ of the propagated state w.r.t. the Lambert arc as a function of time over the full trajectory.

(b) Derive a model to use the results for $\mathbf{\Phi}_i(t_{i+1}, t_i)$ to compute the change in velocity $\Delta\mathbf{v}_i$ needed at the beginning of each arc to ensure that the spacecraft reaches $\bar{\mathbf{r}}$ at the end of the arc. [4] Keep the initial position of the arc constant. Show the derivation of your model. For this question, neglect linearization errors (assume $\Delta\tilde{\mathbf{x}}(t) = \Delta\mathbf{x}(t)$)

(c) Modify the code, so that the initial states of each arc are modified in accordance with your calculated correction manuevers in (b). Plot the deviation $\Delta r(t)$ of the modified propagated state w.r.t. the Lambert arc as a function of time.

(d) Redo (c), but now iterate the differential correction algorithm (separately for each arc). That is, with the corrected trajectories from (c), calculate new correction maneuvers (using the same algorithm!) to further reduce the trajectory mismatches at the arc end times. Terminate the iteration for a given arc when the condition $\mathbf{r}_i = \bar{\mathbf{r}}_i$ is met with a tolerance of better than 1 m (that is: $||(\mathbf{r}_i - \bar{\mathbf{r}}_i|| < 1$ m). Make a table of the number of iterations required per arc, and explain why some arcs require more/less iterations than others.

(e) From (d), you will have a list of $\Delta\mathbf{v}_i$ values (maneuver at the start of each arc) and a list of state histories $\mathbf{x}_i(t)$ for each arc. From these, derive and evaluate an expression for the ΔV required over the full trajectory, which is needed to fly the trajectory (ignore the remaining position mis-match in position between the arcs). In short: if the trajectory you designed under (d) were to be flown, and the dynamical model you used was a perfect representation of reality: how much ΔV would be required?

Add to save file 1: Row 9: initial propagation time and Cartesian state (arc 0), question a.
Add to save file 1: Row 10: final propagation time and Cartesian state (arc 0), question a.
Add to save file 1: Row 11: initial propagation time and Cartesian state (arc 4), question a.
Add to save file 1: Row 12: final propagation time and Cartesian state (arc 4), question a.

[4] For the purposes of this question, fix the arc start and end times to those of question a

4. **(35 points; Maximum text length: 30 lines)**
For this question, you will perform a similar analysis as in the previous question, but now using an approximate model for a low-thrust acceleration to correct the orbit so that it reaches the start and end of the trajectory correctly. Instead of a ten-arc model, you will use a one- and two-arc model.

In your model, you will parameterize the thrust as a constant acceleration in the RSW frame for each arc. This thrust will be added to the simulation by defining a so-called 'empirical' acceleration: a constant acceleration in RSW direction[5]. Note that, since the direction of the RSW frame changes in time, the inertial direction of this empirical acceleration will change in time. For this question, you will compute the magnitudes of these emprical accelerations, such that the trajectory meets all the required boundary conditions.

By calculating the sensitivity matrix $\mathbf{S}$ for the entries of the empirical acceleration, you will be able to calculate (approximately) the required thust under a number of different conditions.

Answer the following questions:

(a) Consider a single arc propagation for the full transfer. Derive an equation to use the results for $\mathbf{S}(t_E)$ to compute the low-thrust acceleration in RSW frame (denote this as $\mathbf{p}$) needed to ensure that the spacecraft reaches $\bar{\mathbf{r}}(t_E)$ at the end of the transfer. Keep the initial state of the arc constant at $\bar{\mathbf{x}}(t_0)$. Do not put any constraints on the final velocity. Show the derivation of your model, starting from the equations in the lecture videos/slides.

(b) Implement your model for question (a) in your code. Use the same value of Δt as in question 2, case iii. Verify and argue that your model works correctly (in the sense that the linearization model works sufficiently well to be useful). Do not perform any iterations of the differential correction.

In the next sub-questions you will analyze a low-thrust trajectory for a two-arc model (with each arc having equal duration). For this question, do not impose any *a priori* constraints on the specific position or velocity values at the arc splitting point (unlike in question 3, where this point was required to correspond to $\bar{\mathbf{r}}$), but instead impose a constraint of continuity in position and velocity over the full trajectory (unlike in question 3, where the velocity was discontinuous between arcs).

(c) Solve for the thrust in the two arcs in the following manner:

i First propagate the dynamics of the first arc and, in doing so, use the value of $\mathbf{p}$ obtained under (b) (denote this as $\mathbf{p}^{(b)}$) for $\mathbf{p}_1$. Use the final state of the first arc as the initial state for the second arc. Derive and present a method to compute the thrust acceleration

[5] In this assignment, you will neglect the reduction in vehicle mass as a result of the use of thrust

$\mathbf{p}_2$ such that the final position of the second arc reaches $\bar{\mathbf{r}}(t_E)$. Write out the equation that you use to compute $\mathbf{p}_2$, and clearly describe the sequence of propagations required to obtain the quantities that are needed as inputs for this equation. Iterate your solution until convergence (to $\bar{\mathbf{r}}(t_E)$) and plot the resulting $\Delta r(t)$ as a function of time over the full trajectory (keeping $\mathbf{p}_1$ fixed).

ii How does the value of $\mathbf{p}_2$ compare to the value of $\mathbf{p}^{(b)}$? What is the underlying reason that the two values are (not) substantially different?

(d) You will now analyze the influence of allowing more freedom in the value of $\mathbf{p}_1$. Do a Monte Carlo[6] analysis for the model from (c), propagating 1,000 times with random variations on the components of $\mathbf{p}_1$, using a Gaussian distribution with $\mathbf{p}^{(b)}$ mean and a standard deviation of $0.4||\mathbf{p}^{(b)}||$ on each component.

i To assess the quality of the resulting orbits, plot $(||\mathbf{p}_1||+||\mathbf{p}_2||)/2$ (the average thrust level), against $||\mathbf{p}_1-\mathbf{p}^{(b)}||$ (the deviation from the initial guess).

ii Plot the thrust acceleration components as a function of time for the model under (b) and the optimal (lowest $||\mathbf{p}_1||+||\mathbf{p}_2||$) found here. Discuss possible reasons for why the solution found here (does not) provide(s) a lower average thrust than the solution in (b) and (c). Relate your answer to the dynamics of the system.

Add to save file 1: Row 13: final propagation time and Cartesian state (question 4b)
Add to save file 1: Row 14: final propagation time and Cartesian state (question 4c)

[6] See e.g. `https://en.wikipedia.org/wiki/Monte_Carlo_method`

Submission and reporting instructions

Reporting instructions - formulating equations of motion

Follow the notation from the lecture notes:

- Use the following indices: S for Sun, E, V, and M, for Earth, Venus, and Mars, respectively, and p for the spacecraft.
- Use a comma to separate indices as you see fit (spacecraft position w.r.t. the Sun can be written as $\mathbf{r}_{S,p}$ or $\mathbf{r}_{Sp}$).
- There is no need to specify the frame orientation of any of the vectors. All are assumed to be in a frame with inertial orientation.
- When denoting separate accelerations, always denote the body B exerting, and body A undergoing, the acceleration with $\mathbf{a}_{BA}$.
- When writing out a single acceleration in terms of positions $\mathbf{r}$, always first write the total relative positions $\mathbf{r}_{CD}$ as used by the acceleration model. Expand the positions further as you see fit in next steps.
- When writing a position $\mathbf{r}_{CD}$ that is partly retrieved from the environment (if any), and where part of the vector is numerically propagated, split the separate contributions (in a second step after the previous point). For instance, if $\mathbf{r}_{ED}$ is propagated, and $\mathbf{r}_{CD}$ is used in the acceleration, write $\mathbf{r}_{CD} = \mathbf{r}_{CE} + \mathbf{r}_{ED}$.

Reporting instructions - change in position

When asked to plot/compute the change in total position between two simulation results $\mathbf{r}_1(t)$ and $\mathbf{r}_2(t)$. The change in total position is to be computed as $||\Delta\mathbf{r}(t)|| = ||\mathbf{r}_2(t) - \mathbf{r}_1(t)||$

Reporting instructions - figures

When using figures, take the following guidelines into account:

- Any text (legend, axis labels, *etc.*) should be sufficiently large so as to be legible when printed on A4 paper.
- Each curve should be distinguishable in your plots - overlapping curves should be distinguished by means of *e.g.* line style, or should be put in a different figure.
- Adjust the scale (*e.g.* linear vs. logarithmic) of your plots as needed to interpret your data.
- Make efficient use of space for plots and other figures. Whenever possible and legible: plot multiple curves (*e.g.* for different runs and/or elements) in a single figure.
- All figures must be complete (including axis labels, legend, caption, *etc.*)

Points will be deducted for unreadable figures, or figures that do not clearly show information that you refer to in your discussion.

Reporting instructions - cover page

The cover page of each report **must contain**:

- A link (as either plain text or a clickable link) to the **private** Github repository containing the source code and output files (see below).
- The names of any people with whom you cooperated (if any)
- The time spent per question

If any of these points are not present on your cover page, points will be deducted.

Reporting instructions - general

Follow the provided limitations on the length of the text that you use (this excludes figures, tables, and equations). Answers longer than the imposed limit will not be read beyond this limitation. Example: if the imposed limitation is 10 lines, and you write 15, we will grade the answer based **only** on the first 10 lines.

You are free to work together with your fellow students, but you are required to write and be able to explain your own code and report. Copying/pasting code or written content from fellow students or AI is strictly prohibited, and can (and has in the past) lead to the case being referred to the Faculty Board of Examiners as a case of fraud. Use of AI is permitted, but should be treated equally to discussions with fellow students: as a source to test ideas, discuss, etc. not as a direct source of code or text.

You are required to keep a log of all AI queries and replies that you make in your preparation of your report, and submit this as a `pdf` along with your report. Failure to do so means that your report cannot be graded. More detailed instructions will be posted on Brightspace.

Points are deducted for having *wrong* statements in the answer to a question. If a question asks you to provide a concrete explanation for a certain phenomenon, and you provide three *possible* explanations, no points will be awarded even if only one out of the three answers that you provided is wrong.

Support instructions

In case of any questions, there are a number of options for support:

- For issues with the installation, or the general use of Tudat, please post an issue on Brightspace. When posting an issue, first browse through existing posts. If your problem is raised in an open issue, post in that issue instead of opening a new one. Mention the operating system you are using.
- In case of questions specific to the assignments, use the Brightspace forum. Go through existing posts before opening a new one. Do not publicly post your code, or other information that provides direct answers to the questions.
- In-person support and Q&A is also available during working lectures and open office hours. See the Brightspace calendar for time and location.

See Brightspace (Course Information → Staff and Support) for details on what to prepare when asking for support.

Submission instructions

You will not be graded on your coding style. Submission of the report and results summary is to be done through Brightspace. **Deadline for submission can be found on Brightspace**. For late submissions, 1 point (out of a total of 10) will be subtracted per day. So, when handing in the report x days late, $\lceil x \rceil$ points will be deducted. *If* you have g grace days left: when handing in the report x days late, $\lceil x - g \rceil$ points will be deducted.

The code submitted to Github must be the exact version used to generate the results in your report. If the submitted script fails to execute or produces numerical results differing from your report/summary file (beyond machine precision), the associated question will not be graded.

Submission of your final code will be done through Github. Ensure that ***you only commit to a private repository***. Instructions on pushing code to Github can be found in:

`https://docs.tudat.space/en/latest/_src_getting_started/_src_use_of_tools/github.html#getting-existing-tudat-code`

Manually uploading the files to Github is not permitted.

Note that we recommend that you regularly commit your modifications to your code. Large single commits (for a large part of the assignment) will be flagged and scrutinized.

See below for the exact files and filenames to submit:

Failure to comply exactly with the requirements for file contents and naming set out below will result in point deductions.

In addition to the report, for this assignment you will submit:

- Five `.py` files (which can be run directly without modifications to reproduce your results), one for each question, and one with the helper functions. Keep the same file names as in the code provided to you.
- To Brightspace: A single `pdf` file with the full log of your usage of AI (detailed instructions to be provided through a Brightspace Announcement)
- A text file containing **only** the initial or final time (column 1) and Cartesian states (as a row vector; columns {2-7}) from a number of your simulations, to at least 8 digits of precision. The specific simulations for which you are to save the time/state, and the row in which you are to save them are indicated in the questions. Name the file `CartesianResults_AE4868_2026_2_YYYYYYY.dat`, where YYYYYYY is your student number. Upload this file to Brightspace.

Failure to comply exactly with the requirements for file contents and naming set out above will result in point

deductions.

Note that you may, but need not, add any other files that you like to your Github repo.